

Speed and Accuracy Comparison for DeterministicQMC

The **mvtnorm** authors

May 3, 2026

1 Purpose

The default **mvtnorm** algorithm, **GenzBretz**, is randomized quasi-Monte Carlo (QMC) (Genz and Bretz, 2002, 2009). Its randomization is one reason it is fast and useful: the algorithm can combine low-discrepancy integration points with a stochastic error assessment. The new **DeterministicQMC** algorithm removes that randomization, so the relevant question is empirical as well as methodological: how much speed is traded for deterministic reproducibility, and how close are the resulting probabilities on representative problems?

This vignette compares eight choices:

- **GenzBretz()**: the current default, with its default stopping tolerances.
- **Miwa(steps = 128)**: the established deterministic low-dimensional normal algorithm.
- original randomized QMC: **GenzBretz(maxpts = 25000, abseps = 0, releps = 0)**, which disables early stopping and uses the historical Genz-Bretz QMC implementation at a fixed point budget.
- **DeterministicQMC(maxpts = 25000, backend = "c")**: the new plain C deterministic Halton backend for normal probabilities.
- **DeterministicQMC(maxpts = 25000, backend = "sobol")**: the new plain C deterministic Sobol backend for normal probabilities.
- **DeterministicQMC(maxpts = 25000, backend = "hybrid")**: an experimental Sobol backend with a deterministic tail-spaced change of variables.

- `DeterministicQMC(maxpts = 25000, backend = "genzsobol")`: an experimental Sobol backend with variable reordering and embedded stability stopping.
- `DeterministicQMC(maxpts = 50000, backend = "auto")`: the automatic deterministic policy, using cross-checked Genz-Sobol when supported and deterministic FORTRAN otherwise.

The “original randomized QMC” row is included separately from the default because the default may stop early once its tolerance is met. The fixed-budget row is often the cleaner comparison to `DeterministicQMC` because both are asked to spend the same nominal number of points. The Auto row is deliberately given a larger budget because it splits the work across deterministic cross-checks; this is a benchmark setting for `abseps = 1e-5`, not a proposed package default.

2 Benchmark design

The examples below use equicorrelated-by-distance Toeplitz correlation matrices, semi-infinite lower bounds, and finite upper bounds. The dimensions are deliberately modest so that package checks remain fast and so that the `Miwa` comparison is still available. Accuracy is measured as relative error against a higher-step `Miwa(steps = 512)` calculation, to five significant digits. This reference is not a proof of the exact probability; it is a stable deterministic yardstick for this small benchmark.

Each reported time is the median elapsed time over three evaluations on the machine building the vignette. Stochastic methods are run with a fixed seed so that the values in the table are reproducible, although the timing itself is necessarily machine dependent.

```
> make_problem <- function(n) {
+   list(corr = toeplitz(0.35^(0:(n - 1))),
+       lower = rep(-Inf, n),
+       upper = seq(-0.2, 0.6, length.out = n))
+ }
> methods <- list(
+   list(name = "Default GenzBretz",
+       algorithm = GenzBretz(),
+       seed = 11L),
+   list(name = "Miwa steps=128",
```

```

+         algorithm = Miwa(steps = 128),
+         seed = NULL),
+   list(name = "Original randomized QMC",
+         algorithm = GenzBretz(maxpts = 25000, abseps = 0, releps = 0),
+         seed = 11L),
+   list(name = "DeterministicQMC C",
+         algorithm = DeterministicQMC(maxpts = 25000, abseps = 0,
+                                       releps = 0, backend = "c"),
+         seed = NULL),
+   list(name = "DeterministicQMC Sobol",
+         algorithm = DeterministicQMC(maxpts = 25000, abseps = 0,
+                                       releps = 0, backend = "sobol"),
+         seed = NULL),
+   list(name = "DeterministicQMC Hybrid",
+         algorithm = DeterministicQMC(maxpts = 25000, abseps = 0,
+                                       releps = 0, backend = "hybrid"),
+         seed = NULL),
+   list(name = "DeterministicQMC GenzSobol",
+         algorithm = DeterministicQMC(maxpts = 25000, abseps = 1e-5,
+                                       releps = 0, backend = "genzsobol"),
+         seed = NULL),
+   list(name = "DeterministicQMC Auto",
+         algorithm = DeterministicQMC(maxpts = 50000, abseps = 1e-5,
+                                       releps = 0, backend = "auto"),
+         seed = NULL)
+ )
> timed_value <- function(problem, method, reps = 3L) {
+   times <- numeric(reps)
+   value <- NA_real_
+   diagnostic <- NA_real_
+   target_requested <- is.list(method$algorithm) &&
+     ((length(method$algorithm$abseps) == 1L &&
+       !is.na(method$algorithm$abseps) && method$algorithm$abseps > 0) ||
+      (length(method$algorithm$releps) == 1L &&
+       !is.na(method$algorithm$releps) && method$algorithm$releps > 0))
+   target_status <- "not requested"
+   for (i in seq_len(reps)) {
+     if (!is.null(method$seed))
+       set.seed(method$seed)
+     invisible(gc(FALSE))

```

```

+       tm <- system.time({
+         ans <- pmvnorm(lower = problem$lower, upper = problem$upper,
+           corr = problem$corr,
+           algorithm = method$algorithm)
+       })
+       times[i] <- unname(tm["elapsed"])
+       if (i == 1L) {
+         value <- as.numeric(ans)
+         diagnostic <- attr(ans, "error")
+         if (target_requested)
+           target_status <- if (identical(attr(ans, "msg"),
+             "Normal Completion"))
+             "met" else "not met"
+       }
+     }
+     list(value = value, diagnostic = diagnostic,
+       target_status = target_status, elapsed = median(times))
+   }
> benchmark_dimension <- function(n) {
+   problem <- make_problem(n)
+   reference <- as.numeric(pmvnorm(lower = problem$lower,
+     upper = problem$upper,
+     corr = problem$corr,
+     algorithm = Miwa(steps = 512)))
+   rows <- lapply(methods, function(method) {
+     ans <- timed_value(problem, method)
+     data.frame(dimension = n,
+       method = method$name,
+       elapsed_sec = sprintf("%.4f", ans$elapsed),
+       value = sprintf("%.8g", ans$value),
+       diagnostic = sprintf("%.3e", ans$diagnostic),
+       target_status = ans$target_status,
+       rel_error = sprintf("%.5g",
+         if (reference == 0) NA_real_ else
+         abs(ans$value - reference) / abs(reference)),
+       stringsAsFactors = FALSE)
+   })
+   do.call(rbind, rows)
+ }
> results <- do.call(rbind, lapply(c(4L, 8L, 12L), benchmark_dimension))

```

```
> print(results, row.names = FALSE)
```

dimension	method	elapsed_sec	value	diagnostic
4	Default GenzBretz	0.0000	0.16701894	1.155e-04
4	Miwa steps=128	0.0000	0.16700578	NA
4	Original randomized QMC	0.0020	0.16700455	5.857e-06
4	DeterministicQMC C	0.0040	0.16702089	2.769e-05
4	DeterministicQMC Sobol	0.0030	0.16700506	4.636e-05
4	DeterministicQMC Hybrid	0.0030	0.16700114	4.554e-05
4	DeterministicQMC GenzSobol	0.0000	0.16700289	3.374e-06
4	DeterministicQMC Auto	0.0040	0.16700364	3.374e-06
8	Default GenzBretz	0.0020	0.036112684	1.662e-05
8	Miwa steps=128	0.0030	0.03639772	NA
8	Original randomized QMC	0.0060	0.036109084	8.721e-06
8	DeterministicQMC C	0.0100	0.036122192	2.726e-05
8	DeterministicQMC Sobol	0.0090	0.036102446	2.816e-05
8	DeterministicQMC Hybrid	0.0100	0.036092146	3.989e-05
8	DeterministicQMC GenzSobol	0.0060	0.036098207	8.347e-06
8	DeterministicQMC Auto	0.0080	0.036102678	9.872e-06
12	Default GenzBretz	0.0130	0.0077504458	3.008e-06
12	Miwa steps=128	0.1620	0.008486779	NA
12	Original randomized QMC	0.0120	0.0077504458	3.008e-06
12	DeterministicQMC C	0.0150	0.0077547872	1.415e-05
12	DeterministicQMC Sobol	0.0150	0.0077452405	1.280e-05
12	DeterministicQMC Hybrid	0.0150	0.0077383798	2.257e-05
12	DeterministicQMC GenzSobol	0.0100	0.0077417723	6.319e-06
12	DeterministicQMC Auto	0.0120	0.0077462859	6.282e-06
target_status rel_error				
	met	7.931e-05		
not requested	5.0514e-07			
not requested	6.8511e-06			
not requested	9.1038e-05			
not requested	3.7882e-06			
not requested	2.7256e-05			
	met	1.6792e-05		
	met	1.23e-05		
	met	0.00068834		
not requested	0.0071992			
not requested	0.00078794			
not requested	0.00042521			

not requested	0.00097164
not requested	0.0012566
met	0.0010889
met	0.0009652
met	0.0156
not requested	0.077923
not requested	0.0156
not requested	0.015049
not requested	0.016261
not requested	0.017133
met	0.016702
met	0.016129

3 Reading the results

The default `GenzBretz()` row answers the practical question “what happens if I use `mvtnorm` as shipped?” The fixed-budget QMC row answers a different question: “what does the original randomized QMC path do if it is given the same nominal point budget as the deterministic C backend?”

For low dimensions, `Miwa` is often competitive and accurate. Its disadvantage is structural rather than visible in every small table: it is a grid-based deterministic method and is limited to normal probabilities in dimension at most 20. Increasing the number of grid steps can improve accuracy but increases cost.

`DeterministicQMC` is intended to occupy a different part of the design space. It gives a deterministic QMC-style calculation under `maxpts` control. This is useful when repeated evaluations must agree bit for bit without managing seeds. The Halton row is the simple first C backend. The Sobol row tests the first stronger deterministic point set in the same transformed integral. The Hybrid row keeps the Sobol backbone but applies a deterministic tail-spaced warp with a Jacobian correction. The trade-off is that the C backends do not provide the mature randomized error assessment or all adaptive refinements of `GenzBretz`. Therefore the benchmark should be read as an implementation baseline, not as a claim that the first C backends are already uniformly faster than the stochastic default. The `GenzSobol` row is allowed to use its deterministic embedded stopping rule, so it answers a slightly different speed question than the fixed-budget C, Sobol, and Hybrid rows. The Auto row adds deterministic cross-checks at fixed sequence offsets and reports the larger of the embedded Genz-Sobol stability errors

and the between-check spread. When a tolerance is requested, those individual checks use a tighter internal target before the combined diagnostic is compared with the requested **abseps/releps**. It is therefore the closest row to the unattended best-practice deterministic policy: somewhat more work is acceptable in exchange for a more conservative reproducible accuracy diagnostic. The larger Auto budget in this benchmark compensates for those repeated cross-checks; it should not be read as a change to the current package defaults.

When **abseps** or **releps** is positive, **DeterministicQMC** treats it as a request that the reported deterministic diagnostic meet the requested target before **maxpts** is exhausted. If the diagnostic is larger than the requested tolerance, the returned object has **inform** = 1 and the usual message "Completion with error > abseps". In the table, **target_status** records whether such a target was requested and met. This is a warning about the diagnostic, not a proof that the true integration error is above or below the target.

4 Sobol, hybrid, Genz-Sobol

The benchmark now includes three implementations of the proposed deterministic Genz-Sobol direction. **backend** = "sobel" keeps the sequential Genz transformation and uses Sobol direction numbers with Gray-code point generation in plain C. **backend** = "hybrid" applies a gentle deterministic tail-spaced change of variables to the Sobol coordinates, with the corresponding Jacobian correction. That makes the nonuniform-grid idea testable without replacing QMC by a full tensor grid. **backend** = "genzsobol" adds marginal-width variable reordering and embedded powers-of-two stability checks using **abseps** and **releps**. **backend** = "auto" wraps that Genz-Sobol backend in a deterministic selection policy: supported normal problems use several fixed sequence offsets and unsupported cases use deterministic FORTRAN rather than falling back to randomized QMC.

The remaining target is not just "use Sobol" or "warp the tails"; it is to add the surrounding algorithmic pieces that make the randomized Genz-Bretz path so effective:

- keep the sequential Genz transformation and the same probability interface;
- reorder variables before integration to reduce effective dimension;

- use a small pilot pass to identify conditional tail regions or high-curvature coordinates before selecting any refinement grid;
- replace the present fixed sequence offsets with stronger deterministic digital shifts when a stability diagnostic is needed;
- use fixed deterministic shifts only as a reproducible stability check, not as randomization;
- keep the FORTRAN deterministic path as a reference and fallback.

The fully developed version would aim at the speed of **GenzBretz** by using fast point generation, embedded refinement, effective-dimension reduction, and selective tail-aware refinement. It would aim at the accuracy of **GenzBretz** by retaining the Genz transformation and using a stronger low-discrepancy rule than the current Halton prototype. The important caveat is that the error attribute would still be deterministic: it could measure convergence and stability across fixed blocks, but it would not be a Monte Carlo confidence statement.

5 Accuracy caveats

There are two different notions of uncertainty in this comparison. The **GenzBretz** error attribute is tied to randomized integration. The **DeterministicQMC** error attribute is a deterministic block-stability diagnostic. These quantities can be numerically useful, but they are not the same statistical object. For applications where stochastic error assessment is important, **GenzBretz** remains the conservative default. For deterministic optimization, testing, or reproducible simulation pipelines, the new backend provides a stable alternative.

Positive **abseps** and **releps** values request deterministic diagnostic error control only. They do not provide a Monte Carlo confidence interval and they do not prove a rigorous deterministic error bound.

References

Genz A, Bretz F (2002). “Methods for the Computation of Multivariate t Probabilities.” *Journal of Computational and Graphical Statistics*, **11**(4), 950–971. doi:10.1198/106186002394.

Genz A, Bretz F (2009). *Computation of Multivariate Normal and t Probabilities*. Lecture Notes in Statistics. Springer-Verlag, Heidelberg, Germany. ISBN 978-3-642-01688-2.